

Critic-Guided Stepwise Distillation for SWE-Bench Agents

Igor Slinko, Ilia Zavidnyi, Egor Bogomolov, Yaroslav Zharov

Rationale and Overview

Problem 1 In LLM agent distillation, successful trajectories are often assumed entirely beneficial, while failed ones are disregarded. However, successful trajectories can contain missteps to avoid, and failed trajectories can include successful steps worth learning from.

Problem 2 In real-world scenarios, trajectories usually lack labels.

Approach We propose *critic-guided, stepwise distillation* that focuses on learning from individual steps within trajectories, rather than accepting or rejecting entire trajectories:

Method For each assistant turn, LLM-based critic assigns labels: *good*, *mistake*, *unnecessary*, or *recover*. In the training of a student model, we mask loss on *mistake* steps, while still preserving them in the input context so that recovery behavior is learnable.

Results We show that our approach consistently improves success rate when the training dataset contains failed trajectories.

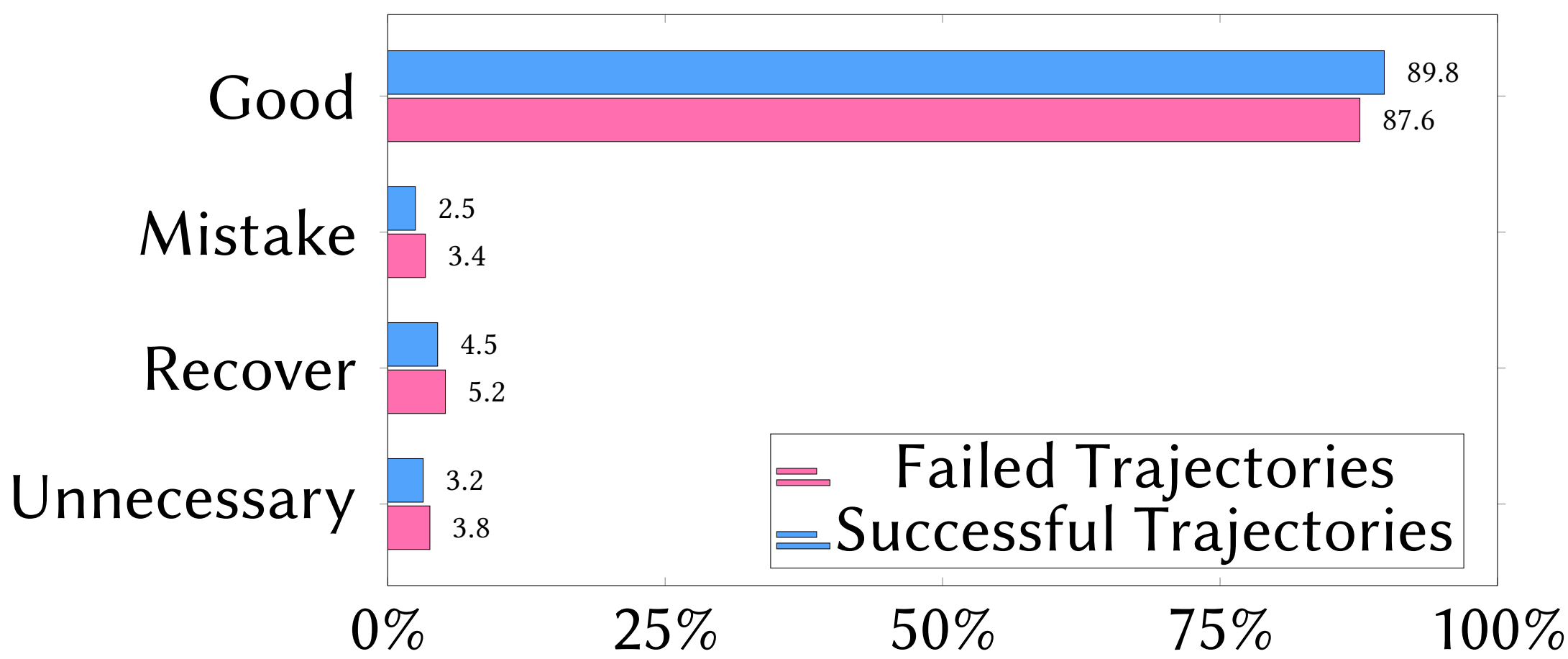


Figure: Label distribution in the training datasets.

Example of the Step-wise Labeled Trajectory

Critic's label:	Trajectory:
	SP: You are a helpful assistant...
	UP: Consider the following PR description...
good	A1: find . -type f -name "*.py"
	O1: ./dvc/tasks.py ./dvc/exceptions.py...
good	A2: view ./dvc/commands/queue/kill.py
	O2: from dvc.cli import formatter...
unnecessary	A3: view ./dvc/repo/experiments/queue/remove.py
	O3: from collections.abc import Collection...
mistake	A4: edit ./dvc/commands/queue/kill.py
	O4: ./dvc/commands/queue/kill.py edited
good	A5: create ./test_queue_kill.py "from dvc.cli..."
	O5: ./test_queue_kill.py created
good	A6: python test_queue_kill.py
	O6: ERROR: failed to init DVC - '.dvc' exists
recover	A7: str_replace getChild -> getLogger
	O7: ./test_queue_kill.py edited
good	A8: python test_queue_kill.py
	O8: DVC repo initialized
good	A9: submit

Trajectory after Critic LLM evaluation. Assistant step #4 was marked as a mistake because the agent started fixing code before reproducing the bug, which directly contradicts the instruction given in the System Prompt (SP). During training, we will only compute loss on assistant steps marked in green.

Experimental Setup and Results

Setup:

- ▶ Critic: Claude 4 Sonnet
- ▶ Student: Qwen2.5-Coder-32B-Instruct
- ▶ Scaffold: SWE-Agent

Dataset: SWE-Smith → 3 splits:

- ▶ 5k successful trajectories
- ▶ 5k failed trajectories
- ▶ 5k successful + 5k failed

Train dataset	Resolved Rate% ↑	pass@7% ↑	Submit Tool% ↑	Steps# ↓
[5k-success]	30.9 ± 1.1	45.8	65.7 ± 1.9	34.7 ± 0.8
[5k success masked]	30.5 ± 1.1	42.4	64.7 ± 1.5	34.1 ± 0.5
[5k fail]	27.7 ± 0.9	42.2	53.6 ± 1.2	39.6 ± 0.7
[5k fail masked]	29.5 ± 1.5	43.8	63.2 ± 1.4	33.5 ± 0.4
[5k success] + [5k fail]	28.5 ± 1.7	43.8	65.3 ± 3.3	36.0 ± 2.1
[5k success] + [5k fail masked]	29.5 ± 1.0	41.8	67.2 ± 1.1	32.9 ± 0.5
[5k success masked] + [5k fail masked]	30.4 ± 1.1	43.6	66.3 ± 2.5	35.4 ± 1.5

Table: Experimental results on SWE-Bench Verified. Each experiment was run 7 times to remove evaluation bias; we show the average and standard deviation. pass@7% is the percentage of successful patches among all 7 rollouts. Submit Tool% is the percentage of trajectories that contained the submit tool call. Steps# is the average number of assistant steps in the trajectories.

On Statistical Significance

Noise Sources:

- ▶ Different rollouts of the agent
- ▶ Randomness in model training

Observed Variability:

- ▶ Up to 4.8% difference in Resolved Rate% between consecutive rollouts
- ▶ Up to 1.4% difference by varying training seed

To estimate the statistical significance we conducted several trainings of the same model and evaluated it multiple times on the [5k failed trajectories] dataset with and without masking.

Train dataset	Resolved Rate% ↑	pass@7% ↑	Submit Tool% ↑	Steps# ↓	Resolved Rate% by Rollout
[5k fail] train #1	28.8 ± 1.0	42.8	58.6 ± 2.7	38.9 ± 2.1	29.6, 29.6, 30.0, 28.2, 28.0, 28.8, 27.4
[5k fail] train #2	27.7 ± 0.9	42.2	53.6 ± 1.2	39.6 ± 0.7	26.2, 28.6, 27.0, 28.8, 27.2, 28.0, 28.2
[5k fail] train #3	29.1 ± 1.0	41.8	60.2 ± 1.0	31.4 ± 1.9	30.4, 29.2, 29.0, 28.8, 30.2, 28.4, 27.6
[5k fail] combined	28.5 ± 1.1	42.3	57.5 ± 3.3	36.6 ± 3.8	
[5k fail masked] train #1	29.8 ± 0.6	42.6	58.4 ± 1.8	42.5 ± 2.3	29.6, 30.4, 30.0, 29.8, 29.6, 28.8, 30.6
[5k fail masked] train #2	29.5 ± 1.5	43.8	63.2 ± 1.4	33.5 ± 2.1	30.0, 29.4, 26.4, 30.8, 29.0, 30.0, 30.8
[5k fail masked] combined	29.7 ± 1.1	43.2	60.8 ± 2.8	38.0 ± 4.6	

Table: Detailed results from multiple model retraining runs showing individual evaluation scores. The Resolved column shows the raw resolved rates for each of the 7 evaluation rollouts.

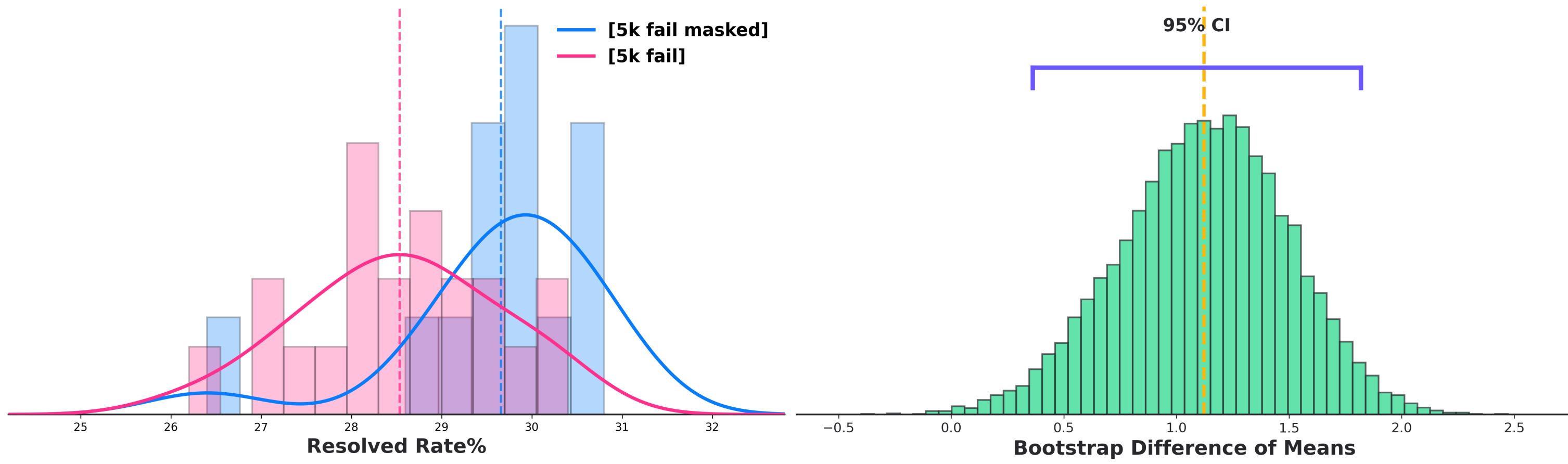


Figure: Distribution analysis of Resolved Rate% for [5k fail] vs [5k fail masked] datasets. Bootstrap analysis reveals a statistically significant improvement of 1.1% with 95% CI [0.4, 1.8].